

Programmierblatt 0: Zahl der Wahl

⇒ Abgabe der Lösungen bis Montag, 06. November 16:00 im AsSESS

In dem ersten Teil der Programmieraufgabe sollen Sie ein Shell-Skript erstellen, welches das Übersetzen des zweiten Aufgabenteils mithilfe des C-Compilers übernimmt. In dem zweiten Aufgabenteil soll eine 5-stellige Zufallszahl mit möglichst wenig Rateversuchen gefunden werden.

Organisatorisches

- Die Aufgaben sind *in Dreiergruppen* zu bearbeiten. Der Lösungsweg und die Programmierung sind gemeinsam zu erarbeiten.
- Die Programmieraufgaben müssen jeweils montags bis 16 Uhr abgegeben werden. In den darauffolgenden Tafelübungen werden die Lösungen der Aufgaben besprochen.
- Die abgegebenen Antworten/Programme werden automatisch auf Ähnlichkeit mit anderen Abgaben überprüft. Wer beim Abschreiben¹ erwischt wird, bekommt ohne weitere Vorwarnung für dieses Blatt keine Punkte.
- Die Aufgaben sind über AsSESS (<https://ess.cs.uos.de/ASSESS/>) abzugeben. Dort gibt *ein* Gruppenmitglied die erforderlichen Dateien ab und nennt dabei die anderen beteiligten Gruppenmitglieder (Matrikelnummer, Vor- und Nachname erforderlich!). Namen und Anzahl der abzugebenden C-Quelldateien² variieren und stehen in der jeweiligen Aufgabenstellung; Antworten auf Theorie- bzw. Textfragen sind grundsätzlich gesammelt in der Datei `antworten.txt`³ zu beantworten. Bis zum Abgabetermin kann eine Aufgabe beliebig oft abgegeben werden – es gilt die letzte vor der Abgabefrist vorgenommene Abgabe. Dabei müssen immer *alle* Dateien hochgeladen werden.
- Sobald eine Abgabe von den Betreuern korrigiert wurde, kann die korrigierte Lösung ebenfalls im AsSESS eingesehen werden.

Geben Sie Ihre Entwicklungsschritte jeweils in den angegebenen Dateien ab.

1 Pseudonym (0 Punkte)

Überlegen Sie sich für Ihre Gruppe ein Pseudonym. Die Ergebnisse werden unter dem Pseudonym auf der Besteliste (<https://ess.cs.uos.de/teaching/BS/bestenliste.html.de>) veröffentlicht.

2 Shell-Skript (40 Punkte)

Um den Übersetzungsprozess von Aufgabe 3 zu vereinfachen, soll ein Shell-Skript erstellt werden. Im späteren Verlauf der Vorlesung werden wir noch andere Tools kennenlernen, um das Übersetzen zu

¹Da wir im Regelfall nicht unterscheiden können, wer von wem abgeschrieben hat, gilt das für Original **und** Plagiat.

²codiert in UTF-8

³reine Textdatei, codiert in UTF-8

vereinfachen. Das Skript „make.sh“ soll mit zwei unterschiedlichen Argumenten aufgerufen werden können: „compile“ und „clean“.

Wurde „compile“ als Argument übergeben, soll Aufgabe 3 mithilfe von gcc übersetzt werden (s. Aufgabe 3) und das gebaute Programm soll in einen lokalen Ordner „bin“ gespeichert werden. Hierfür muss der bin Ordner erstellt werden, falls er noch nicht existiert. Für das Übersetzen sollen die Flags „-std=c11 -Wall -Wextra -Werror“ genutzt werden. Außerdem soll das Programm nur übersetzt werden, wenn die Quelltextdatei ein neueres Änderungsdatum als die Programm-Datei hat. (25 Punkte)

Ist „clean“ als Argument gewählt worden, soll das gebaute Programm inklusive des „bin“-Ordners gelöscht werden. Achten Sie darauf nur den **lokalen** Ordner zu löschen und nicht den „bin“-Ordner im Rootverzeichnis des Systems! (10 Punkte)

Wenn weder „compile“ noch „clean“ oder gar kein Argument übergeben wurde, soll sich das Skript mit einer geeigneten Fehlermeldung beenden. (5 Punkte)

Hinweise:

- Um ein Shell-Skript ausführbar zu machen muss das Execute-Flag mithilfe von **chmod(1)** für die Datei gesetzt werden.
- Um Strings oder die Zeitstempel von Dateien zu vergleichen, lässt sich **test(1)** verwenden.

→ make.sh

3 Zahl der Wahl (20 + 20 + 20 Punkte)

Schon wieder die Zahlenkombination vom Fahrradschloss vergessen? Kein Problem. In dieser Aufgabe üben wir, 5-stellige Zufallszahlen zu erraten. Im Gegensatz zum Fahrradschloss haben wir hier allerdings die praktische Hilfestellung, dass wir für jeden Rateversuch überprüfen können, ob die geschätzte Zahl kleiner oder größer als die tatsächliche Kombination ist (oder ob wir richtig geraten haben). Unser Ziel ist es, mit möglichst wenigen Versuchen, die Zahlenkombination zu erraten.

Die grundsätzliche Architektur des Spiels ist schon vorgegeben. Sie besteht aus drei Dateien:

1. **guess.c** - Hier soll die Funktion 'unsigned int guess_the_number(void)' implementiert werden, ohne dabei die Signatur der Funktion zu verändern. Es soll lediglich der Funktionsrumpf ausgefüllt werden. Es ist erlaubt, Hilfsfunktionen zu definieren und zu verwenden. (Vorsicht - Vorwärtsdeklaration notwendig!) Nur diese Datei soll für diese Aufgabe eingereicht werden. Die Funktion soll die gesuchte Zahl zurückgeben.
2. **main.c** - Hier werden die Zufallszahlen erzeugt. Diese Datei sollte nicht verändert und nicht abgegeben werden. Bei Ausführung der main() Funktion in der Datei werden nacheinander 100 Zufallszahlen erzeugt. Für jede Zufallszahl wird genau einmal die Funktion guess_the_number(void) [siehe oben] aufgerufen. Zusätzlich enthält die Datei noch eine Funktion 'int evaluate(unsigned int)'. Diese sollte an einer beliebigen Stelle aus der Datei guess.c aufgerufen werden, um zu prüfen, ob die gesuchte Zahl mit der geratenen Zahl übereinstimmt. In diesem Fall gibt die evaluate-Funktion den Wert 0 zurück. Ist die gesuchte Zahl kleiner als die geratene Zahl, gibt die Funktion -1 zurück. Ist die gesuchte Zahl größer als die geratene Zahl, gibt die Funktion 1 zurück. Es wird außerdem mitgezählt, wie häufig die Funktion evaluate aufgerufen wird.
3. **guessing.h** - Diese Datei deklariert die beiden Funktionen guess_the_number(void) und evaluate(unsigned int). Die beiden Dateien guess.c und main.c binden diese Datei ein. So können die beiden Funktionen aus beiden Dateien heraus genutzt werden. Die Datei soll nicht verändert und nicht abgegeben werden.

Die Dateien `main.c`, `guess.h` und `guessing.h` können genutzt werden, um das gesamte Spiel zu testen. Dazu kann das Spiel mit dem Skript aus Aufgabe 1 kompiliert werden. Alternativ lässt sich der folgende Befehl verwenden:

```
gcc -std=c11 -Wall -Wextra -Werror -o game main.c guess.c
```

Danach kann es mit dem folgenden Befehl ausgeführt werden:

```
./game
```

Momentan ist der Zufallszahlengenerator so eingestellt, dass er immer wieder die gleiche Serie an Zufallszahlen generiert (Seed: 42). Bei der Bewertung wird der Seed natürlich ein anderer sein.

Bei dieser Aufgabe sind auch 'unkonventionelle' Lösungsmethoden erlaubt, solange ihr zeigen könnt, dass ihr die Aufgabe auch auf konventionelle Art lösen könntet (z.B. mit einer auskommentierten Lösung). Als 'unkonventionelle' Lösungen zählen alle Lösungen, wo keine echte Suche nach der Zahl statt findet.

Bei der Aufgabe wird überprüft:

- ob die Funktion `guess_the_number(void)` zuverlässig die gesuchte Zahl findet (20 Punkte)
- ob die durchschnittliche Anzahl an Versuchen kleiner als 50 ist (weitere 20 Zusatzpunkte)
- ob die durchschnittliche Anzahl an Versuchen kleiner als 20 ist (weitere 20 Zusatzpunkte)

→ `guess.c`

Tipps zu den Programmieraufgaben:

- Kommentieren Sie Ihren Quellcode ausführlich, so dass wir auch bei Programmierfehlern im Zweifelsfall noch Punkte vergeben können!
- Achten Sie darauf, dass sich der Programmcode ohne Warnungen übersetzen lässt, z.B. durch Nutzung von `-Werror`.